

As a Senior Database Architect, I have designed the schema for the Banking Management System, prioritizing transactional integrity, auditability, and scalability.

---

## 1. Database Overview

The database uses a relational model (RDBMS) to ensure ACID compliance for financial operations. It is structured to separate identity management, core banking services, and audit trails.

## 2. ER Diagram Description

- **User/Auth Core:** Centralizes users, roles, and MFA tokens.
- **Banking Core:** Manages accounts and transactions.
- **Loan/Workflow:** Manages application states and staff assignments.
- **Analytics/Fraud:** Stores AI insights and fraud logs.
- **Audit Trail:** Immutable logs linked to system actors.

---

## 3. Tables & Schema

### Core User & Identity

- **Users:** `user\_id` (PK), `email`, `password\_hash`, `role\_id` (Admin/Employee/Customer), `mfa\_secret`, `status`.
- **Roles:** `role\_id` (PK), `role\_name`.

### Banking

- **Accounts:** `account\_id` (PK), `user\_id` (FK), `balance`, `currency`, `account\_type`.
- **Transactions:** `transaction\_id` (PK), `from\_account` (FK), `to\_account` (FK), `amount`, `timestamp`, `status`, `is\_flagged`.

### Loans

- 
- **LoanApplications:** `loan\_id` (PK), `user\_id` (FK), `amount`, `monthly\_income`, `status`, `processed\_by` (FK - Employee ID).

### Fraud & Insights

- **FraudLogs:** `log\_id` (PK), `transaction\_id` (FK), `reason`, `severity`, `status`.
- **AllInsights:** `insight\_id` (PK), `user\_id` (FK), `recommendation\_text`, `generated\_at`.

### Audit

- **AuditLogs:** `audit\_id` (PK), `user\_id` (FK), `action`, `table\_affected`, `old\_value`, `new\_value`, `timestamp`.

---

## 4. SQL Schema (PostgreSQL Syntax)

```

CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    role VARCHAR(50) NOT NULL,
    mfa_enabled BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE accounts (
    account_id SERIAL PRIMARY KEY,
    user_id INT REFERENCES users(user_id),
    balance DECIMAL(15, 2) DEFAULT 0.00,
    currency VARCHAR(3) DEFAULT 'USD',
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE transactions (
    transaction_id UUID PRIMARY KEY,
    from_account INT REFERENCES accounts(account_id),
    to_account INT REFERENCES accounts(account_id),
    amount DECIMAL(15, 2) NOT NULL,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(20),
    is_flagged BOOLEAN DEFAULT FALSE
);

CREATE TABLE loan_applications (
    loan_id SERIAL PRIMARY KEY,
    user_id INT REFERENCES users(user_id),
    requested_amount DECIMAL(15, 2),
    status VARCHAR(20) DEFAULT 'SUBMITTED', -- SUBMITTED, PENDING, APPROVED, REJECTED
    processed_by INT REFERENCES users(user_id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE audit_logs (
    audit_id SERIAL PRIMARY KEY,
    user_id INT REFERENCES users(user_id),
    action TEXT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

---

## 5. Constraints & Indexes

- **Constraints:**

---

``CHK_LoanLimit``: Trigger-based check ensuring ``requested_amount` <= (monthly_income 5)`.

\* ``FK_Transactions``: Prevents deletion of accounts with existing transaction history.

- **Indexes:**

\* ``idx_user_email``: Optimized login performance.

\* ``idx_transaction_date``: Optimized reporting for audit and fraud detection.

\* ``idx_loan_status``: Optimized for Employee application queues.

---

## 6. Normalization Notes

- **3NF Compliance:** All tables are in Third Normal Form.
- **Data Redundancy:** Minimal; customer profiles are normalized out of the ``Accounts`` table.
- **Auditability:** ``AuditLogs`` are designed to be append-only, ensuring the "Immutable logs" requirement is met.
- **Financial Precision:** ``DECIMAL(15,2)`` is used instead of ``FLOAT`` to prevent floating-point rounding errors common in financial calculations.

## 7. Recommendations

1. **Partitioning:** Partition the ``Transactions`` table by ``timestamp`` (Monthly) to ensure the 2-second performance requirement as data grows.
2. **Encryption:** Apply Column-Level Encryption on ``balance`` and ``email`` fields to satisfy security requirements.
3. **Soft Deletes:** Use ``is_active`` flags instead of ``DELETE`` commands to maintain data integrity for regulatory audits.